

# Assignment 5.3: Analyzing Sales Data with Hive on Hadoop and DuckDB

Kalyani Guggilla

DTAN 515: Big Data

Indiana Wesleyan University

September 27, 2026

# 1. Introduction

The goal of this assignment was to load a real sales dataset (SalesJan2009.csv) into the Hadoop ecosystem, query it with Apache Hive, and interpret the results. The required query finds the number of products sold in each country. I also adapted two queries from the Hortonworks SQL-to-Hive cheat sheet, and I repeated all three queries in DuckDB, the alternative lab environment suggested for this assignment.

Hadoop stores large files across a cluster with the Hadoop Distributed File System (HDFS) and processes them in parallel with MapReduce. Hive sits on top of Hadoop and lets analysts write familiar SQL (HiveQL), which Hive turns into MapReduce jobs. Hue is a web interface for running these queries. DuckDB is a lightweight SQL engine that runs on a single computer and is useful when a full Hadoop cluster is not available.

## 2. Project Purpose and Business Questions

The purpose of this project is to turn one month of raw transaction data into information a business could act on, while showing how Hive on Hadoop can answer business questions with SQL. I imagined the role of an analyst for the online retailer that produced this data, who needs to decide where to focus marketing, which payment options matter most and how far the results can be trusted. The analysis answers four key business questions:

Table 1. Key business questions and how each one is answered.

| # | Business question                                                                                    | How it is answered                                                                  | Why it matters                                      |
|---|------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|-----------------------------------------------------|
| 1 | Where are our products sold, and how concentrated are sales by country?                              | Query 1: products sold in each country (required query)                             | Shows which markets to protect and which to grow    |
| 2 | How do customers pay, and which payment methods bring the most revenue and the highest average sale? | Query 2: revenue by payment type (GROUP BY, SUM, AVG)                               | Guides checkout design and card-specific promotions |
| 3 | Within the largest market, the United States, which states buy the most?                             | Query 3: top 10 US states (WHERE, ORDER BY, LIMIT)                                  | Helps target regional advertising                   |
| 4 | Can the results be trusted?                                                                          | Row-count check (998 rows, header excluded) and the same queries repeated in DuckDB | Wrong counts would lead to wrong business decisions |

### 3. Environment Setup

The free Hadoop Starter Kit cluster was difficult to access, so I built a single-node Hadoop environment on my own MacBook Air and ran Hive through its command-line client, Beeline, instead of the Hue web interface. The same HiveQL statements would run unchanged in Hue.

| Component                | Version / setting                           |
|--------------------------|---------------------------------------------|
| Hadoop (HDFS, MapReduce) | 3.5.0, installed with Homebrew              |
| HDFS address             | hdfs://localhost:9000, replication factor 1 |
| Apache Hive / Beeline    | 4.2.1, running on Java 21                   |
| Hive execution engine    | MapReduce in local mode                     |
| DuckDB (alternative lab) | 1.5.5 command-line client                   |

### 4. Data Preparation

The dataset has 998 credit-card transactions from January 2009 with 12 columns: Transaction\_date, Product, Price, Payment\_Type, Name, City, State, Country, Account\_Created, Last\_Login, Latitude and Longitude. I first confirmed the file was on my computer (123,637 bytes).



**Figure 2.** Cleaning the file and uploading it to HDFS.

Next, I created an external Hive table that points to the HDFS folder. I used the OpenCSVSerde reader so that quoted values containing commas are handled correctly, and I told Hive to skip the header line. Because the table is external, dropping it would not delete the data in HDFS.

```
CREATE EXTERNAL TABLE sales (  
  transaction_date STRING, product STRING, price STRING, payment_type STRING,  
  name STRING, city STRING, state STRING, country STRING,  
  account_created STRING, last_login STRING, latitude STRING, longitude STRING)  
ROW FORMAT SERDE 'org.apache.hadoop.hive.serde2.OpenCSVSerde'  
STORED AS TEXTFILE  
LOCATION '/user/chinnu/sales'  
TBLPROPERTIES ("skip.header.line.count"="1");
```

## Verifying the Row Count and Header Handling

Before running any analysis, I verified that the table contained exactly the 998 transactions and that the header line was not being counted as a sale. The raw file has 999 lines: one header line and 998 transactions. (The `wc -l` command reported 998 because it counts line breaks, and the last line of the file has no line break at the end.) So the correct table must return 998 rows.

The first time I ran `SELECT COUNT(*) FROM sales`, Hive returned 999. The `skip.header.line.count` setting was working for simple `SELECT` statements, but the MapReduce job that performs `COUNT(*)` used a combined input format that ignored it and counted the header as a data row. I fixed this by switching Hive to the standard `HiveInputFormat`. These are the exact statements that produced the verified count of 998:

```
-- Session settings (run each time Beeline is opened)  
SET hive.execution.engine=mr;  
SET mapreduce.framework.name=local;  
SET hive.input.format=org.apache.hadoop.hive.ql.io.HiveInputFormat;  
  
-- Row-count verification  
SELECT COUNT(*) FROM sales;           -- result: 998  
  
-- Header check: the first row returned must be a sale, not the header  
SELECT * FROM sales LIMIT 3;           -- first row: 1/2/09 6:17, Product1, 1200,  
Mastercard
```

Table 2. Row-count and header verification steps.

| Step                                             | Exact query                                | Result                                 | What it shows                                |
|--------------------------------------------------|--------------------------------------------|----------------------------------------|----------------------------------------------|
| 1. Default Hive settings                         | SELECT COUNT(*) FROM sales;                | 999                                    | The header line was counted as a sale        |
| 2. After SET hive.input.format = HiveInputFormat | SELECT COUNT(*) FROM sales;                | 998                                    | Header skipped; matches the 998 transactions |
| 3. Sample rows                                   | SELECT * FROM sales LIMIT 3;               | First row: 1/2/09 6:17, Product1, 1200 | No header text appears as data               |
| 4. DuckDB (independent check)                    | SELECT COUNT(*) AS total_sales FROM sales; | 998                                    | A second engine confirms the count           |

Figure 3 shows both results in the same session: the incorrect 999 at the top, followed by the SET statement and the corrected count of 998.

```

+-----+
| 999 |
+-----+
1 row selected (1.617 seconds)
26/09/27 12:12:37 [dcd64123-b540-4800-8589-4b50b4e6d82d main]: WARN calcite.RelOptHiveTable: No Stats for default@sales, Columns: transaction_date, country, product, payment_type, city, price, last_login, lat
itude, name, account_created, state, longitude
No Stats for default@sales, Columns: transaction_date, country, product, payment_type, city, price, last_login, latitude, name, account_created, state, longitude
26/09/27 12:12:37 [dcd64123-b540-4800-8589-4b50b4e6d82d main]: WARN optimizer.SimpleFetchOptimizer: Table default@sales is external table, falling back to filesystem scan.
+-----+
| sales.transaction_date | sales.product | sales.price | sales.payment_type | sales.name | sales.city | sales.state | sales.country | sales.account_created | sales.last_logi
n | sales.latitude | sales.longitude |
+-----+
| 1/2/09 6:17 | Product1 | 1200 | Mastercard | carolina | Basildon | England | United Kingdom | 1/2/09 6:00 | 1/2/09 6:08
| 51.5 | -1.1166667 | 1200 | Visa | Betina | Parkville | MO | United States | 1/2/09 4:42 | 1/2/09 7:49
| 39.195 | -94.68394 | 1200 | Mastercard | Federica e Andrea | Astoria | OR | United States | 1/1/09 16:21 | 1/3/09 12:32
| 1/2/09 13:08 | Product1 | 1200 | Mastercard | Federica e Andrea | Astoria | OR | United States | 1/1/09 16:21 | 1/3/09 12:32
| 46.18806 | -123.83 |
+-----+
3 rows selected (0.095 seconds)
0: jdbc:hive2://> SET hive.input.format=org.apache.hadoop.hive.q1.io.HiveInputFormat;
> SELECT COUNT(*) FROM sales;
No rows affected (0.005 seconds)
26/09/27 12:14:14 [HiveServer2-Background-Pool: Thread-201]: WARN ql.Driver: Hive-on-MR is deprecated in Hive 2 and may not be available in the future versions. Consider using a different execution engine (i.
e. tez) or using Hive 1.X releases.
Query ID = chinmu_20260927121414_e8448c3f-54ba-486a-9936-3724406cae5f
Total jobs = 1
Launching Job 1 out of 1
Number of reduce tasks determined at compile time: 1
In order to change the average load for a reducer (in bytes):
  set hive.exec.reducers.bytes.per.reducer=<number>
In order to limit the maximum number of reducers:
  set hive.exec.reducers.max=<number>
In order to set a constant number of reducers:
  set mapreduce.job.reduces=<number>
26/09/27 12:14:14 [HiveServer2-Background-Pool: Thread-201]: WARN impl.MetricsSystemImpl: JobTracker metrics system already initialized!
26/09/27 12:14:14 [HiveServer2-Background-Pool: Thread-201]: WARN impl.MetricsSystemImpl: JobTracker metrics system already initialized!
26/09/27 12:14:14 [HiveServer2-Background-Pool: Thread-201]: WARN mapreduce.JobResourceUploader: Hadoop command-line option parsing not performed. Implement the Tool interface and execute your application wit
h ToolRunner to remedy this.
Job running in-process (local Hadoop)
26/09/27 12:14:14 [pool-17-thread-1]: WARN impl.MetricsSystemImpl: JobTracker metrics system already initialized!
WARN : Hive-on-MR is deprecated in Hive 2 and may not be available in the future versions. Consider using a different execution engine (i.e. tez) or using Hive 1.X releases.
2026-09-27 12:14:15,819 Stage-1 map = 100%, reduce = 100%
Ended Job = job_local1868598330_0002
MapReduce Jobs Launched:
Stage-Stage-1: HDFS Read: 502742 HDFS Write: 0 HDFS EC Read: 0 SUCCESS
Total MapReduce CPU Time Spent: 0 msec
+-----+
| _c0 |
+-----+
| 998 |
+-----+
1 row selected (1.343 seconds)
0: jdbc:hive2://> SET hive.execution.engine=mr;
> SET mapreduce.framework.name=local;
> SET hive.input.format=org.apache.hadoop.hive.q1.io.HiveInputFormat;
26/09/27 12:14:17 [dcd64123-b540-4800-8589-4b50b4e6d82d main]: WARN processors.SetProcessor: Hive-on-MR is deprecated in Hive 2 and may not be available in the future versions. Consider using a different exec
ution engine (i.e. tez) or using Hive 1.X releases.
No rows affected (0.000 seconds)
No rows affected (0.005 seconds)
No rows affected (0.004 seconds)
0: jdbc:hive2://>

```

**Figure 3.** Hive output showing the first count (999), the sample rows and the verified count of 998 after the header fix.

## 5. Hive Queries and Results

### Query 1 (required): Number of Products Sold in Each Country

```
SELECT TRIM(country) AS country, COUNT(*) AS products_sold
FROM sales
GROUP BY TRIM(country)
ORDER BY products_sold DESC;
```

Hive ran this as two MapReduce jobs: the first grouped and counted the rows, and the second sorted the totals. The query returned 56 countries, and their counts add up to exactly 998. TRIM removes extra spaces so the same country is never counted twice.

```
0: jdbc:hive2://> SELECT TRIM(country) AS country, COUNT(*) AS products_sold
> FROM sales
> GROUP BY TRIM(country)
> ORDER BY products_sold DESC;
26/09/27 12:22:02 [dc644123-b540-4800-8509-4b50b4e6d82d main]: WARN calcite.RelOptHiveTable: No Stats for default@sales, Columns: country
No Stats for default@sales, Columns: country
26/09/27 12:22:02 [HiveServer2-Background-Pool: Thread-271]: WARN ql.Driver: Hive-on-MR is deprecated in Hive 2 and may not be available in the future versions. Consider using a different execution engine (i.
e. tez) or using Hive 1.X releases.
Query ID = chinnu_20260927122202_71f07b72-09f2-4b24-a7ef-ef97eb33af42
Total Jobs = 2
Launching Job 1 out of 2
Number of reduce tasks not specified. Estimated from input data size: 1
In order to change the average load for a reducer (in bytes):
  set hive.exec.reducers.bytes.per.reducer=<number>
In order to limit the maximum number of reducers:
  set hive.exec.reducers.max=<number>
In order to set a constant number of reducers:
  set mapreduce.job.reducers=<number>
26/09/27 12:22:02 [HiveServer2-Background-Pool: Thread-271]: WARN impl.MetricsSystemImpl: JobTracker metrics system already initialized!
26/09/27 12:22:02 [HiveServer2-Background-Pool: Thread-271]: WARN impl.MetricsSystemImpl: JobTracker metrics system already initialized!
26/09/27 12:22:02 [HiveServer2-Background-Pool: Thread-271]: WARN mapreduce.JobResourceUploader: Hadoop command-line option parsing not performed. Implement the Tool interface and execute your application wit
h ToolRunner to remedy this.
Job running in-process (local Hadoop)
26/09/27 12:22:02 [pool-20-thread-1]: WARN impl.MetricsSystemImpl: JobTracker metrics system already initialized!
WARN : Hive-on-MR is deprecated in Hive 2 and may not be available in the future versions. Consider using a different execution engine (i.e. tez) or using Hive 1.X releases.
2026-09-27 12:22:05,756 Stage-1 map = 100%, reduce = 100%
Ended Job = job_local1115241341_0003
Launching Job 2 out of 2
Number of reduce tasks determined at compile time: 1
In order to change the average load for a reducer (in bytes):
  set hive.exec.reducers.bytes.per.reducer=<number>
In order to limit the maximum number of reducers:
  set hive.exec.reducers.max=<number>
In order to set a constant number of reducers:
  set mapreduce.job.reducers=<number>
26/09/27 12:22:03 [HiveServer2-Background-Pool: Thread-271]: WARN impl.MetricsSystemImpl: JobTracker metrics system already initialized!
26/09/27 12:22:03 [HiveServer2-Background-Pool: Thread-271]: WARN impl.MetricsSystemImpl: JobTracker metrics system already initialized!
26/09/27 12:22:03 [HiveServer2-Background-Pool: Thread-271]: WARN mapreduce.JobResourceUploader: Hadoop command-line option parsing not performed. Implement the Tool interface and execute your application wit
h ToolRunner to remedy this.
Job running in-process (local Hadoop)
26/09/27 12:22:04 [pool-13-thread-1]: WARN impl.MetricsSystemImpl: JobTracker metrics system already initialized!
2026-09-27 12:22:05,006 Stage-2 map = 100%, reduce = 100%
Ended Job = job_local1569041956_0004
MapReduce Jobs Launched:
Stage-Stage=1: HDFS Read: 750018 HDFS Write: 0 HDFS EC Read: 0 SUCCESS
Stage-Stage=2: HDFS Read: 750018 HDFS Write: 0 HDFS EC Read: 0 SUCCESS
Total MapReduce CPU Time Spent: 0 msec
+-----+
| country | products_sold |
+-----+
| United States | 463 |
| United Kingdom | 100 |
| Canada | 76 |
| Ireland | 49 |
| Australia | 38 |
| Switzerland | 36 |
| France | 27 |
| Germany | 25 |
| Netherlands | 22 |
| Norway | 16 |
| Denmark | 15 |
| Italy | 15 |
| Sweden | 13 |
| Spain | 12 |
| Belgium | 8 |
+-----+
```

**Figure 4.** Query 1 results in Hive (top of the list).

Stage-Stage-1: HDFS Read: 750018 HDFS Write: 0 HDFS EC Read: 0 SUCCESS  
 Stage-Stage-2: HDFS Read: 750018 HDFS Write: 0 HDFS EC Read: 0 SUCCESS  
 Total MapReduce CPU Time Spent: 0 msec

| country              | products_sold |
|----------------------|---------------|
| United States        | 463           |
| United Kingdom       | 100           |
| Canada               | 76            |
| Ireland              | 49            |
| Australia            | 38            |
| Switzerland          | 36            |
| France               | 27            |
| Germany              | 25            |
| Netherlands          | 22            |
| Norway               | 16            |
| Denmark              | 15            |
| Italy                | 15            |
| Sweden               | 13            |
| Spain                | 12            |
| Belgium              | 8             |
| Austria              | 7             |
| United Arab Emirates | 6             |
| Turkey               | 6             |
| New Zealand          | 6             |
| Brazil               | 5             |
| South Africa         | 5             |
| Hungary              | 3             |
| Czech Republic       | 3             |
| Japan                | 2             |
| The Bahamas          | 2             |
| Thailand             | 2             |
| Poland               | 2             |
| Philippines          | 2             |
| Monaco               | 2             |
| Malta                | 2             |
| India                | 2             |
| Finland              | 2             |
| Ukraine              | 1             |
| Iceland              | 1             |
| Kuwait               | 1             |
| Hong Kong            | 1             |
| Guatemala            | 1             |
| Greece               | 1             |
| Latvia               | 1             |
| Luxembourg           | 1             |
| Argentina            | 1             |
| Dominican Republic   | 1             |
| Malaysia             | 1             |
| Mauritius            | 1             |
| Costa Rica           | 1             |
| China                | 1             |
| Cayman Isls          | 1             |
| Moldova              | 1             |
| Bulgaria             | 1             |
| Romania              | 1             |
| Bermuda              | 1             |
| Russia               | 1             |
| Bahrain              | 1             |
| South Korea          | 1             |
| Israel               | 1             |
| Jersey               | 1             |

56 rows selected (2.712 seconds)  
 0: jdbc:hive2://>

**Figure 5.** Query 1 results in Hive (end of the list, 56 rows selected).

Table 3. Top five countries by products sold (from Query 1).

| Country              | Products sold | Share of all sales |
|----------------------|---------------|--------------------|
| United States        | 463           | 46.4%              |
| United Kingdom       | 100           | 10.0%              |
| Canada               | 76            | 7.6%               |
| Ireland              | 49            | 4.9%               |
| Australia            | 38            | 3.8%               |
| Other 51 countries   | 272           | 27.3%              |
| Total (56 countries) | 998           | 100%               |

## Query 2 (cheat-sheet pattern: GROUP BY with SUM and AVG): Revenue by Payment Type

This query adapts the cheat sheet's aggregation pattern (SELECT with GROUP BY, SUM and AVG). The Price column was stored as text, and some prices contain commas (for example, "1,200"), so I removed the commas with REPLACE and converted the values to numbers with CAST before adding them up.

```
SELECT payment_type,
       COUNT(*) AS transactions,
```

```

SUM(CAST(REPLACE(price, ',', '') AS DOUBLE)) AS total_revenue,
ROUND(AVG(CAST(REPLACE(price, ',', '') AS DOUBLE)), 2) AS avg_price
FROM sales
GROUP BY payment_type
ORDER BY total_revenue DESC;

```

```

| South Korea | 1 |
| Israel | 1 |
| Jersey | 1 |
+-----+
56 rows selected (2.712 seconds)
0: jdbc:hive2://> SELECT payment_type,
> COUNT(*) AS transactions,
> SUM(CAST(REPLACE(price, ',', '') AS DOUBLE)) AS total_revenue,
> ROUND(AVG(CAST(REPLACE(price, ',', '') AS DOUBLE)), 2) AS avg_price
> FROM sales
> GROUP BY payment_type
> ORDER BY total_revenue DESC;
26/09/27 12:33:18 [ddcd6423-b540-4800-8589-4b50b4e6d82d main]: WARN calcite.RelOptHiveTable: No Stats for default@sales, Columns: payment_type, price
No Stats for default@sales, Columns: payment_type, price
26/09/27 12:33:18 [HiveServer2-Background-Pool: Thread-396]: WARN ql.Driver: Hive-on-MR is deprecated in Hive 2 and may not be available in the future versions. Consider using a different execution engine (i.e. tez) or using Hive 1.X releases.
Query ID = chinmu_20260927123318_0ab99fc6-009b-47d7-8550-4257612e6f7b
Total jobs = 2
Launching Job 1 out of 2
Number of reduce tasks not specified. Estimated from input data size: 1
In order to change the average load for a reducer (in bytes):
  set hive.exec.reducers.bytes.per.reducer=<number>
In order to limit the maximum number of reducers:
  set hive.exec.reducers.max=<number>
In order to set a constant number of reducers:
  set mapreduce.job.reduces=<number>
26/09/27 12:33:18 [HiveServer2-Background-Pool: Thread-396]: WARN impl.MetricsSystemImpl: JobTracker metrics system already initialized!
26/09/27 12:33:18 [HiveServer2-Background-Pool: Thread-396]: WARN impl.MetricsSystemImpl: JobTracker metrics system already initialized!
26/09/27 12:33:18 [HiveServer2-Background-Pool: Thread-396]: WARN mapreduce.JobResourceUploader: Hadoop command-line option parsing not performed. Implement the Tool interface and execute your application with h ToolRunner to remedy this.
Job running in-process (local Hadoop)
26/09/27 12:33:18 [pool-26-thread-1]: WARN impl.MetricsSystemImpl: JobTracker metrics system already initialized!
WARN : Hive-on-MR is deprecated in Hive 2 and may not be available in the future versions. Consider using a different execution engine (i.e. tez) or using Hive 1.X releases.
2026-09-27 12:33:19,726 Stage-1 map = 100%, reduce = 100%
Ended Job = job_local1610591046_0005
Launching Job 2 out of 2
Number of reduce tasks determined at compile time: 1
In order to change the average load for a reducer (in bytes):
  set hive.exec.reducers.bytes.per.reducer=<number>
In order to limit the maximum number of reducers:
  set hive.exec.reducers.max=<number>
In order to set a constant number of reducers:
  set mapreduce.job.reduces=<number>
26/09/27 12:33:19 [HiveServer2-Background-Pool: Thread-396]: WARN impl.MetricsSystemImpl: JobTracker metrics system already initialized!
26/09/27 12:33:19 [HiveServer2-Background-Pool: Thread-396]: WARN impl.MetricsSystemImpl: JobTracker metrics system already initialized!
26/09/27 12:33:19 [HiveServer2-Background-Pool: Thread-396]: WARN mapreduce.JobResourceUploader: Hadoop command-line option parsing not performed. Implement the Tool interface and execute your application with h ToolRunner to remedy this.
Job running in-process (local Hadoop)
26/09/27 12:33:20 [pool-29-thread-1]: WARN impl.MetricsSystemImpl: JobTracker metrics system already initialized!
2026-09-27 12:33:20,996 Stage-2 map = 100%, reduce = 100%
Ended Job = job_local175152701_0006
MapReduce Jobs Launched:
Stage-Stage-1: HDFS Read: 997294 HDFS Write: 0 HDFS EC Read: 0 SUCCESS
Stage-Stage-2: HDFS Read: 997294 HDFS Write: 0 HDFS EC Read: 0 SUCCESS
Total MapReduce CPU Time Spent: 0 msec

```

| payment_type | transactions | total_revenue | avg_price  |
|--------------|--------------|---------------|------------|
| Visa         | 522          | \$849,350.0   | \$1,627.11 |
| Mastercard   | 277          | \$458,450.0   | \$1,655.05 |
| Amex         | 110          | \$188,900.0   | \$1,717.27 |
| Diners       | 89           | \$133,800.0   | \$1,503.37 |

```

4 rows selected (2.657 seconds)
0: jdbc:hive2://>

```

**Figure 6.** Query 2 results in Hive: revenue by payment type.

**Table 4.** Revenue by payment type (from Query 2).

| Payment type | Transactions | Total revenue | Share of revenue | Average sale |
|--------------|--------------|---------------|------------------|--------------|
| Visa         | 522          | \$849,350     | 52.1%            | \$1,627.11   |
| Mastercard   | 277          | \$458,450     | 28.1%            | \$1,655.05   |
| Amex         | 110          | \$188,900     | 11.6%            | \$1,717.27   |
| Diners       | 89           | \$133,800     | 8.2%             | \$1,503.37   |
| Total        | 998          | \$1,630,500   | 100%             | \$1,633.77   |

## Query 3 (cheat-sheet pattern: WHERE, ORDER BY, LIMIT): Top 10 US States

This query adapts the cheat sheet's filtering and sorting pattern. WHERE keeps only United States sales, GROUP BY counts them by state, ORDER BY sorts them from highest to lowest, and LIMIT returns the top 10.

```
SELECT TRIM(state) AS state, COUNT(*) AS products_sold
FROM sales
WHERE TRIM(country) = 'United States'
GROUP BY TRIM(state)
ORDER BY products_sold DESC
LIMIT 10;
```

```
0: jdbc:hive2://> SELECT TRIM(state) AS state, COUNT(*) AS products_sold
> FROM sales
> WHERE TRIM(country) = 'United States'
> GROUP BY TRIM(state)
> ORDER BY products_sold DESC
> LIMIT 10;
26/09/27 12:35:21 [dc64123-b540-4808-8589-4b50b4e6d82d main]: WARN calcite.RelOptHiveTable: No Stats for default@sales, Columns: country, state
No Stats for default@sales, Columns: country, state
26/09/27 12:35:21 [HiveServer2-Background-Pool: Thread-517]: WARN ql.Driver: Hive-on-MR is deprecated in Hive 2 and may not be available in the future versions. Consider using a different execution engine (i.
e. tez) or using Hive 1.X releases.
Query ID = chinnu_20260927123521_6929f5d2-5143-4eb8-9b33-cc4c1ebbee33
Total jobs = 2
Launching Job 1 out of 2
Number of reduce tasks not specified. Estimated from input data size: 1
In order to change the average load for a reducer (in bytes):
  set hive.exec.reducers.bytes.per.reducer=<number>
In order to limit the maximum number of reducers:
  set hive.exec.reducers.max=<number>
In order to set a constant number of reducers:
  set mapreduce.job.reduces=<number>
26/09/27 12:35:21 [HiveServer2-Background-Pool: Thread-517]: WARN impl.MetricsSystemImpl: JobTracker metrics system already initialized!
26/09/27 12:35:21 [HiveServer2-Background-Pool: Thread-517]: WARN impl.MetricsSystemImpl: JobTracker metrics system already initialized!
26/09/27 12:35:21 [HiveServer2-Background-Pool: Thread-517]: WARN mapreduce.JobResourceUploader: Hadoop command-line option parsing not performed. Implement the Tool interface and execute your application wit
h ToolRunner to remedy this.
Job running in-process (local Hadoop)
26/09/27 12:35:21 [pool-32-thread-1]: WARN impl.MetricsSystemImpl: JobTracker metrics system already initialized!
WARN : Hive-on-MR is deprecated in Hive 2 and may not be available in the future versions. Consider using a different execution engine (i.e. tez) or using Hive 1.X releases.
2026-09-27 12:35:22,713 Stage-1 map = 100%,  reduce = 100%
Ended Job = job_local172798519_0007
Launching Job 2 out of 2
Number of reduce tasks determined at compile time: 1
In order to change the average load for a reducer (in bytes):
  set hive.exec.reducers.bytes.per.reducer=<number>
In order to limit the maximum number of reducers:
  set hive.exec.reducers.max=<number>
In order to set a constant number of reducers:
  set mapreduce.job.reduces=<number>
26/09/27 12:35:22 [HiveServer2-Background-Pool: Thread-517]: WARN impl.MetricsSystemImpl: JobTracker metrics system already initialized!
26/09/27 12:35:22 [HiveServer2-Background-Pool: Thread-517]: WARN impl.MetricsSystemImpl: JobTracker metrics system already initialized!
26/09/27 12:35:22 [HiveServer2-Background-Pool: Thread-517]: WARN mapreduce.JobResourceUploader: Hadoop command-line option parsing not performed. Implement the Tool interface and execute your application wit
h ToolRunner to remedy this.
Job running in-process (local Hadoop)
26/09/27 12:35:23 [pool-35-thread-1]: WARN impl.MetricsSystemImpl: JobTracker metrics system already initialized!
2026-09-27 12:35:23,975 Stage-2 map = 100%,  reduce = 100%
Ended Job = job_local177514177_0008
MapReduce Jobs Launched:
Stage-Stage-1:  HDFS Read: 1244570 HDFS Write: 0 HDFS EC Read: 0 SUCCESS
Stage-Stage-2:  HDFS Read: 1244570 HDFS Write: 0 HDFS EC Read: 0 SUCCESS
Total MapReduce CPU Time Spent: 0 msec

+-----+
| state | products_sold |
+-----+
| CA    | 66             |
| NY    | 41             |
| TX    | 37             |
| VA    | 30             |
| FL    | 29             |
| GA    | 21             |
| NJ    | 19             |
| IL    | 16             |
| MD    | 15             |
| WA    | 14             |
+-----+
10 rows selected (2.673 seconds)
0: jdbc:hive2://>
```

**Figure 7.** Query 3 results in Hive: top 10 US states.

## 6. Alternative Lab: The Same Queries in DuckDB

As the alternative lab for part b, I loaded the same cleaned file into DuckDB and ran the same three queries. DuckDB read the CSV in a single command, detected the header and column types automatically, and returned each result almost instantly.

```
CREATE TABLE sales AS SELECT * FROM read_csv_auto('sales_clean.csv', header=true);
SELECT COUNT(*) AS total_sales FROM sales; -- returns 998
```

```
##### 100.0%

Successfully installed DuckDB 1.5.5 to /Users/chinnu/.duckdb/cli/1.5.5/duckdb
Updated symlink from /Users/chinnu/.duckdb/cli/latest/duckdb to
/Users/chinnu/.duckdb/cli/1.5.5/duckdb

Hint: Append the following line to your shell profile:
export PATH="/Users/chinnu/.duckdb/cli/latest:$PATH"

To launch DuckDB 1.5.5 now, type
/Users/chinnu/.duckdb/cli/latest/duckdb
chinnu@Kalyanis-MacBook-Air hive-stocks % cd ~
~/duckdb/cli/latest/duckdb
DuckDB v1.5.5 (Variegata)
Enter <help> for usage hints.
memory D CREATE TABLE sales AS SELECT * FROM read_csv_auto('sales_clean.csv', header=true);
memory D
memory D SELECT COUNT(*) AS total_sales FROM sales;
```

| total_sales |
|-------------|
| 998         |

```
memory D
memory D SELECT TRIM(Country) AS country, COUNT(*) AS products_sold
FROM sales
GROUP BY 1
ORDER BY 2 DESC
LIMIT 15;
```

| country<br>varchar | products_sold<br>int64 |
|--------------------|------------------------|
| United States      | 463                    |
| United Kingdom     | 100                    |
| Canada             | 76                     |
| Ireland            | 49                     |
| Australia          | 38                     |
| Switzerland        | 36                     |
| France             | 27                     |
| Germany            | 25                     |
| Netherlands        | 22                     |
| Norway             | 16                     |
| Denmark            | 15                     |
| Italy              | 15                     |
| Sweden             | 13                     |
| Spain              | 12                     |
| Belgium            | 8                      |

15 rows 2 columns

```
memory D
memory D SELECT Payment_Type,
COUNT(*) AS transactions,
SUM(CAST(REPLACE(CAST(Price AS VARCHAR), ',', '' ) AS DOUBLE)) AS total_revenue,
ROUND(AVG(CAST(REPLACE(CAST(Price AS VARCHAR), ',', '' ) AS DOUBLE)), 2) AS avg_price
FROM sales
GROUP BY 1
ORDER BY 3 DESC;
```

| Payment_Type<br>varchar | transactions<br>int64 | total_revenue<br>double | avg_price<br>double |
|-------------------------|-----------------------|-------------------------|---------------------|
| Visa                    | 522                   | 849350.0                | 1627.11             |
| Mastercard              | 277                   | 458450.0                | 1655.05             |

**Figure 8.** DuckDB: loading the data, the row count and Query 1.

```
GROUP BY 1
ORDER BY 2 DESC
LIMIT 15;
```

| country<br>varchar | products_sold<br>int64 |
|--------------------|------------------------|
| United States      | 463                    |
| United Kingdom     | 100                    |
| Canada             | 76                     |
| Ireland            | 49                     |
| Australia          | 38                     |
| Switzerland        | 36                     |
| France             | 27                     |
| Germany            | 25                     |
| Netherlands        | 22                     |
| Norway             | 16                     |
| Denmark            | 15                     |
| Italy              | 15                     |
| Sweden             | 13                     |
| Spain              | 12                     |
| Belgium            | 8                      |

15 rows 2 columns

```
memory D
memory D SELECT Payment_Type,
COUNT(*) AS transactions,
SUM(CAST(REPLACE(CAST(Price AS VARCHAR), ',', '' ) AS DOUBLE)) AS total_revenue,
ROUND(AVG(CAST(REPLACE(CAST(Price AS VARCHAR), ',', '' ) AS DOUBLE)), 2) AS avg_price
FROM sales
GROUP BY 1
ORDER BY 3 DESC;
```

| Payment_Type<br>varchar | transactions<br>int64 | total_revenue<br>double | avg_price<br>double |
|-------------------------|-----------------------|-------------------------|---------------------|
| Visa                    | 522                   | 849350.0                | 1627.11             |
| Mastercard              | 277                   | 458450.0                | 1655.05             |
| Amex                    | 110                   | 188900.0                | 1717.27             |
| Diners                  | 89                    | 133800.0                | 1503.37             |

```
memory D
memory D SELECT TRIM(State) AS state, COUNT(*) AS products_sold
FROM sales
WHERE TRIM(Country) = 'United States'
GROUP BY 1
ORDER BY 2 DESC
LIMIT 10;
```

| state<br>varchar | products_sold<br>int64 |
|------------------|------------------------|
| CA               | 66                     |
| NY               | 41                     |
| TX               | 37                     |
| VA               | 30                     |
| FL               | 29                     |
| GA               | 21                     |
| NJ               | 19                     |
| IL               | 16                     |
| MD               | 15                     |
| WA               | 14                     |

10 rows 2 columns

```
memory D
```

**Figure 9.** DuckDB: Queries 2 and 3.

Every DuckDB result matched Hive exactly: 998 sales; the United States with 463, the United Kingdom with 100 and Canada with 76; Visa with 522 transactions and \$849,350; and California with 66 sales. Because two independent engines produced identical numbers, the results are verified.

*Table 5. Hive and DuckDB produce identical results.*

| Check             | Hive (Hadoop)  | DuckDB         |
|-------------------|----------------|----------------|
| Total rows        | 998            | 998            |
| US / UK / Canada  | 463 / 100 / 76 | 463 / 100 / 76 |
| Visa revenue      | \$849,350      | \$849,350      |
| Amex average sale | \$1,717.27     | \$1,717.27     |
| CA / NY / TX      | 66 / 41 / 37   | 66 / 41 / 37   |

## 7. Interpretation of Results

### Sales Are Concentrated in a Few English-Speaking Markets

The clearest finding from Query 1 is concentration. The United States alone accounts for 463 of the 998 sales (46.4%), almost half of all business. Together with the United Kingdom (100) and Canada (76), the top three countries make up 639 sales, or 64.0% of the total. Adding Ireland and Australia, the top five, all English-speaking, reach 726 sales (72.7%). This suggests the company's website, marketing and customer support are most effective with English-speaking customers, or that those markets were targeted first.

At the other end is a long tail. Of the 56 countries, 24 recorded only one sale and 33 recorded two or fewer. These markets are too small to judge on one month of data, but together they show that the product reaches customers worldwide without any local marketing. A reasonable business strategy would be to protect and grow the core US, UK and Canadian markets while testing low-cost campaigns in the mid-sized European markets such as Switzerland (36), France (27) and Germany (25), which already show steady demand.

## Visa Drives Most Revenue, but Amex Customers Spend More

Query 2 shows that Visa is the dominant payment method: 522 transactions (52.3%) and \$849,350 in revenue (52.1%) out of a total of \$1,630,500. Visa and Mastercard together account for over 80% of revenue, so a smooth checkout for these two cards is essential. However, the averages tell a different story from the totals. American Express customers have the highest average sale (\$1,717.27), about 5.1% above the overall average of \$1,633.77, while Diners Club customers have the lowest (\$1,503.37). This suggests Amex users are more likely to buy higher-priced products, so they could be a good target for premium offers or promotions. The average sale is also far above the \$1,200 base price of Product1, which means some customers are choosing more expensive products.

## US Demand Follows Population

Query 3 shows that within the United States, California leads with 66 sales (14.3% of US sales), followed by New York (41), Texas (37), Virginia (30) and Florida (29). These are mostly the country's largest and most economically active states, so sales appear to follow population. The top 10 states account for 288 of the 463 US sales (62.2%). For a US campaign, focusing advertising on California, New York and Texas would reach the largest share of existing customers.

## Limitations of the Data

- **Short time period:** the data covers only January 2009, so it cannot show seasonal trends or growth over time.
- **Small sample:** with 998 rows, countries with one or two sales could change a lot from month to month, so conclusions about small markets are uncertain.
- **Data-quality issues:** the file needed cleaning before analysis. It had old line endings, extra spaces in some city names and prices stored as text with commas. Without TRIM, REPLACE and CAST, some results would have been wrong or impossible to calculate.
- **Counts are not revenue:** Query 1 counts transactions, not dollars, so a country with fewer but more expensive sales could be more valuable than its rank suggests.

## Hive Compared with DuckDB

Both tools use almost the same SQL, but they are built for different scales. Hive translated each query into MapReduce jobs that read from HDFS, and each query took about 2.7 seconds even on this small file, mostly overhead from starting the jobs. That overhead is worthwhile when the data is terabytes spread across many machines, because Hive can process it in parallel with fault tolerance. DuckDB runs on one computer and answered the same queries almost instantly, which makes it the better choice for a file of this size. The lesson is to choose the tool that fits the size of the data: DuckDB for small and medium files, and Hive on Hadoop for truly big data.

## 8. Challenges and Solutions

| Challenge                                                                   | Solution                                                                                       |
|-----------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|
| The Hadoop Starter Kit cluster was not accessible.                          | Installed Hadoop 3.5.0 and Hive 4.2.1 on my MacBook and ran Hive through Beeline.              |
| Terminal commands typed at the Beeline prompt were not recognized.          | Learned to check the prompt: % for Terminal, jdbc:hive2://> for Hive, D for DuckDB.            |
| Beeline could not open the Hive metastore.                                  | Started Beeline from the ~/hive-stocks folder, where the embedded metastore_db is stored.      |
| Queries failed with a missing Tez class.                                    | Hive 4 defaults to the Tez engine, so I switched to MapReduce in local mode with SET commands. |
| COUNT(*) returned 999 instead of 998.                                       | Set hive.input.format to HiveInputFormat so the header row is skipped by MapReduce jobs.       |
| The file used old line endings, and some values had extra spaces or commas. | Converted line endings with Perl and used TRIM, REPLACE and CAST in the queries.               |

## 9. Conclusion

This assignment showed the full big-data workflow on a small scale: preparing raw data, storing it in HDFS, defining a schema with Hive, querying it with SQL that runs as MapReduce jobs, and checking the results with a second tool. The analysis answered the four business questions set out in Section 2:

- **Where are products sold?** Sales are heavily concentrated: the United States accounts for 46.4% of sales, and the five English-speaking markets together account for 72.7%.
- **How do customers pay?** Visa drives 52.1% of revenue, while American Express customers spend the most per sale (\$1,717.27).
- **Which US states lead?** California, New York and Texas, the largest states, lead US sales, and the top 10 states make up 62.2% of US sales.
- **Can the results be trusted?** Yes. The verified count of 998 rows excludes the header, and DuckDB produced identical results for every query.

Solving the setup problems along the way also gave me practical experience with how Hive, HDFS and MapReduce work together.

## References

Apache Software Foundation. (n.d.). Apache Hive. <https://hive.apache.org/>

Cloudera. (n.d.). Using Hue. Cloudera Runtime 7.0.1 documentation. <https://docs.cloudera.com/runtime/7.0.1/using-hue/topics/hue-using.html>

DuckDB Foundation. (n.d.). DuckDB documentation. <https://duckdb.org/docs/>

Hortonworks. (n.d.). SQL to Hive cheat sheet [PDF]. Course materials, DTAN 515, Indiana Wesleyan University.

Hue. (n.d.). Hue: The open-source SQL assistant for data warehouses. <https://gethue.com/>

Jain, V. K. (2017). Big data and Hadoop. Khanna Publishing. [https://books.google.com/books/about/Big\\_Data\\_and\\_Hadoop.html?id=i6NODQAAQBAJ](https://books.google.com/books/about/Big_Data_and_Hadoop.html?id=i6NODQAAQBAJ)